

stm32f103 workspace notes

Q V2 & Q V3: sharing a pin, then synchronizing it

Two small modifications to the same program - first making each blink's on/off explicit, then making the whole blink exclusive with a mutex - and what each one actually changes about the program's behaviour.

Semaphore APIs used here

JUST THE THREE THAT Q V3 NEEDS

A FreeRTOS mutex is a specific kind of semaphore - a lock with exactly one owner at a time, created and used through three calls.

CALL	WHAT IT DOES
<code>xSemaphoreCreateMutex()</code>	Creates the lock itself, returned as a <code>SemaphoreHandle_t</code> . Starts "available" - the first task to ask for it gets it immediately.
<code>xSemaphoreTake(handle, ticks)</code>	Asks for the lock. If it's free, returns immediately with the lock held. If it's not, the calling task moves to <i>Blocked</i> for up to <code>ticks</code> (<code>portMAX_DELAY</code> = wait forever) rather than spinning.
<code>xSemaphoreGive(handle)</code>	Releases the lock. If another task is Blocked waiting for it, that task becomes Ready immediately. For a mutex specifically, this must be called by the same task that took it.

Why a mutex and not a plain semaphore. A mutex adds two things a generic binary semaphore doesn't have: priority inheritance (irrelevant here, since both tasks share one priority) and an ownership rule - only the task that took it can give it back. That ownership rule is exactly the right shape for "protect this resource," which is what both Q V3's problem and its fix are about.

Q V2 - making on/off explicit

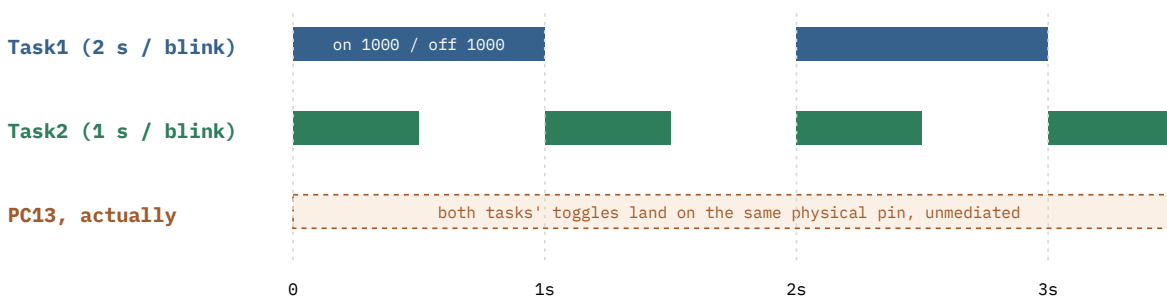
SAME TOGGLE RATE, DIFFERENT LOOP SHAPE

Q V1's loop toggled once per iteration, so "on" and "off" only existed implicitly, as alternating states across iterations. Q V2 restructures each task's loop to spell out a complete blink - on, delay, off, delay - in a single iteration.

shared-led-no-sync/main.c - both tasks, Q V2 shape

```
static void task1_toggle_1hz(void *pvParameters)
{
    for (;;) {
        GPIOC->ODR ^= (1U << 13);          // on
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);          // off
        vTaskDelay(pdMS_TO_TICKS(1000));
    }                                         // one iteration = one 2 s blink
}

static void task2_toggle_2hz(void *pvParameters)
{
    for (;;) {
        GPIOC->ODR ^= (1U << 13);          // on
        vTaskDelay(pdMS_TO_TICKS(500));
        GPIOC->ODR ^= (1U << 13);          // off
        vTaskDelay(pdMS_TO_TICKS(500));
    }                                         // one iteration = one 1 s blink
}
```



the race from Q V1 is still here - Q V2 only changed how the loop body reads, not whether the two tasks' toggles can land in the same instant

The toggle rate is identical to Q V1 - this is purely a structural rewrite so each iteration is a complete, readable blink instead of half of one.

Q V3 - a mutex around the whole blink

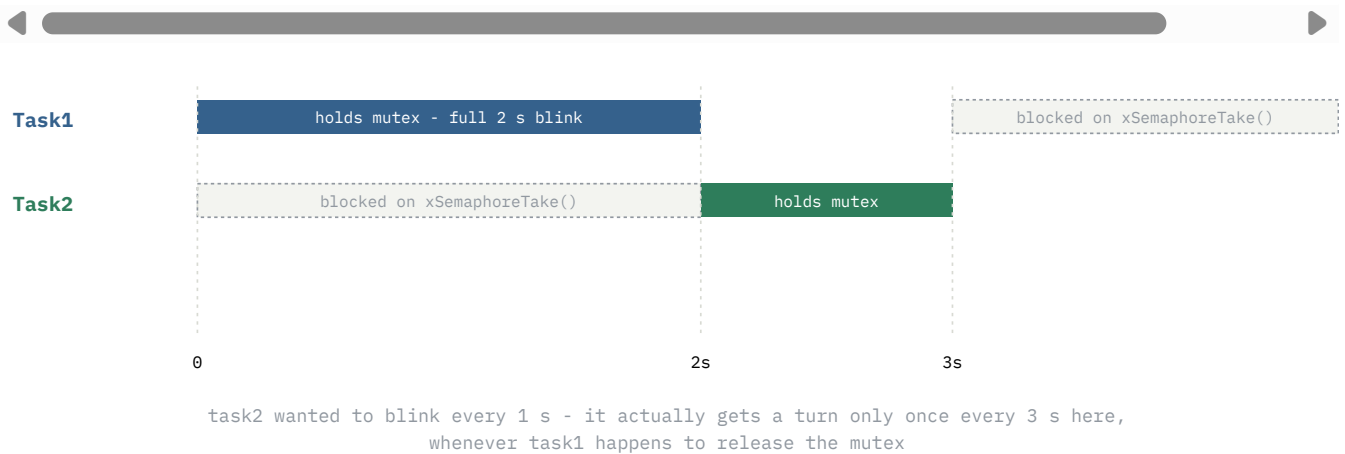
THE RACE IS GONE - BUT SO IS THE INDEPENDENT TIMING

Q V3 wraps each task's *entire* on-delay-off-delay sequence in
`xSemaphoreTake()` / `xSemaphoreGive()` - not just the individual toggles.

shared-led-mutex/main.c - task1 (task2 is the same shape, 500 ms delays)

```
static void task1_toggle_1hz(void *pvParameters)
{
    for (;;) {
        xSemaphoreTake(xLedMutex, portMAX_DELAY);
        GPIOC->ODR ^= (1U << 13);          // on
        vTaskDelay(pdMS_TO_TICKS(1000));
        GPIOC->ODR ^= (1U << 13);          // off
        vTaskDelay(pdMS_TO_TICKS(1000));
        xSemaphoreGive(xLedMutex);
    }
}
```

// mutex held for the FULL 2 s, not just e



The lost-toggle race is completely gone - but neither task's own 1Hz/2Hz timing survives contact with the other once the whole operation, not just the shared register access, is what gets locked.

The trade-off, stated plainly

- **What's fixed:** the non-atomic `ODR` read-modify-write from Q V1/V2 can no longer be interrupted mid-sequence by the other task - correctness on the shared register is total.
- **What's given up:** the two tasks no longer run independently at all. Whichever one grabs the mutex first keeps the other blocked for its *entire* cycle, so they end up alternating whole blinks rather than interleaving on their own schedules.
- **The narrower alternative:** a mutex (or just `GPIOC->BSRR`, which sets/clears a single bit atomically in one write) around only the toggle itself, not the surrounding delays, would remove the same race while leaving each task free to keep its own timing.

stm32f103 workspace notes · continues from the shared-led-no-sync / shared-led-mutex notes